

Day 8 questions: wire, reg and reset

Entries 123, 127 and 129 are Done in the tracker. These notes cover wire and reg, shift-register reset, and BCD clock rollover. Entry 123 is on Day 7; entries 127 and 129 are on Day 8. Reviewed 7 September 2026.

Entry 129: why can't these wires be reg?

```
wire [15:0] aLower = a[15:0];  
// i cant make them reg why
```

This declaration creates a continuously driven connection. Whenever the lower half of `a` changes, `aLower` follows it. The equals sign here belongs to a net declaration assignment; it does not mean an assignment executed once at startup.

```
wire [15:0] aLower;  
assign aLower = a[15:0]; // same connection
```

Changing only wire to reg changes the meaning of the declaration: a variable declaration initializer is a startup initialization, not a continuously tracking expression. If `a` changes later, the initializer does not execute again. An initially unknown input can also leave that variable unknown. The signal needs either a net connection or an explicit procedural assignment that runs when the inputs change.

```
reg [15:0] aLower;  
always @(*) begin  
    aLower = a[15:0];  
end
```

This alternative describes combinational logic because every execution assigns `aLower` and the sensitivity covers the expression inputs. The keyword `reg` itself does not demand a physical register. In this particular adder, the wire form is simpler: no clock or storage is required. You can also connect the input slice directly to the instance port.

```
add16 low (.a(a[15:0]), .b(b[15:0]), .cin(1'b0),  
          .sum(sum[15:0]), .cout(coutL));
```

For example, with `a = 32'h1234_ABCD`, `aLower` is `ABCD`. Changing `a` to `32'h5678_0001` makes the wire become `0001`. A variable initialized from the old input has no such update mechanism.

Source: [Intel Quartus: variable declaration initialization](#)

Source: [HDLBits: Adder 2](#)

Entry 129: why is carry a wire?

```
wire [16:0] carry; // what it is to be wire
assign carry[0] = cin;
assign cout = carry[16];
```

The carry vector is the wiring between full-adder instances. carry[0] receives the external carry-in. Full adder 0 drives carry[1], full adder 1 drives carry[2], and so on until full adder 15 drives carry[16]. Each stage reads a carry from its neighbor. The parent does not assign these connections inside a clocked always block.

In Verilog, use nets such as wire for connections driven by continuous assignments or child-module outputs. Each bit here has one intended driver. A 16-bit adder needs 17 carry connection points: one input plus the 16 stage outputs. Their widths count connections, not extra storage bits.

```
genvar i;
generate
    for (i=0; i<16; i=i+1) begin : stage
        add1 fa (.a(a[i]), .b(b[i]), .cin(carry[i]),
                .sum(sum[i]), .cout(carry[i+1]));
    end
endgenerate
```

Your commented-out version uses $i < 15$. That creates stages 0 through 14 only, leaving bit 15 and carry[16] without the required final stage. A generate loop creates hardware instances during elaboration; it does not wait for 16 clock cycles. A ripple adder has a combinational propagation path through those instances.

For a small trace, consider $011 + 001$ with carry-in 0. Bit 0 computes $1+1+0$, so sum[0]=0 and carry[1]=1. Bit 1 computes $1+0+1$, so sum[1]=0 and carry[2]=1. Bit 2 computes $0+0+1$, giving sum[2]=1 and carry[3]=0. The result is 100.

HDLBits already provides add16 in this exercise. Keep your experimental implementation commented out and submit top_module plus add1. Use named ports when reusing an adder: the Fadd exercise declares cout before sum, while this add1 interface declares sum before cout. Positional connections must match the actual declaration exactly.

SystemVerilog permits a single continuous or module-output driver for variables in cases where classic Verilog requires a net. That does not make a declaration initializer continuous, nor does it make mixing procedural and continuous drivers valid. For these Verilog exercises, wire for structural connections is the clearest choice.

Source: [Intel Quartus: continuous assignment requires a net in Verilog](#)

Entry 127: why not assign q = 0?

```
// assign q = 4'd0; why i cant do that
```

A continuous assignment drives its target all the time. It is not a one-time reset. Even if q were a wire, this statement would permanently drive zero, so the output could never display loaded data or shift. In the current module, q is also written inside an always block; adding another driver conflicts with that ownership. Classic Verilog additionally requires the continuous-assignment target to be a net, whereas your q is declared reg.

An asynchronous reset is part of the flip-flop behavior: asserting areset clears q immediately, even between clock edges. Deasserting reset lets the existing state hold until the next rising clock edge. At that edge, load has priority over ena. If neither is active, the register retains its previous value.

```
always @(posedge clk or posedge areset) begin
    if (areset)
        q <= 4'b0000;
    else if (load)
        q <= data;
    else if (ena)
        q <= {1'b0, q[3:1]};
end
```

For a trace, load data=1011 at a rising edge: q becomes 1011. At the next enabled rising edge, q becomes 0101; the following enabled edge produces 0010. If ena=0 and load=0, q stays 0010. If reset rises between edges, q immediately becomes 0000. A permanently driven zero could not produce the first three states.

The missing final else intentionally means hold for a clocked register; it does not infer a latch. q <= q is unnecessary here. Nonblocking assignments model the clocked state update, so the right-hand side q[3:1] uses the pre-edge value.

A register initializer is a separate concept from an external reset. Initial values may be supported by particular FPGA flows, but they do not respond when areset changes during operation. Keep the reset logic in the event-controlled block as required by the problem.

Source: [HDLBits: asynchronous reset, synchronous load and shift enable](#)

Source: [Intel Quartus: continuous assignment target type](#)

Entry 123: should seconds be reset here?

Final accepted submission | Verified 6 September 2026 in Chrome: **Status: Success!**

The question in the submitted code

```
if (minutes == 8'd59 && second == 8'd59) begin
    minutes <= 8'd0;
    // will i make seconds 0 here ?
end
```

Answer

No. Do not assign second again in the minute block. The seconds block already schedules second `<= 8'h00` when the old value is 59.

Why both blocks work on the same clock edge

Every right-hand side in a clocked nonblocking assignment reads the pre-edge state. At 12:34:59, the seconds logic sees old seconds = 59 and schedules 00. The minute logic also sees old seconds = 59 and schedules minutes = 35. Both scheduled values take effect together after the always block finishes.

Register	Value before edge	Condition seen	Value after edge
second	8'h59	second == 8'h59	8'h00
minutes	8'h34	second == 8'h59	8'h35
hour	8'h12	minutes != 8'h59	8'h12
pmreg	0 (AM)	no 11-to-12 carry	0 (AM)

Why a second assignment is worse

Keep the seconds update in one place. Within the same clocked process, if two nonblocking assignments to second execute, the later one wins. Across separate processes, conflicting writes can race and create multiple drivers.

BCD constant clarification

Use `8'h59` for packed BCD 59. Its two nibbles are 5 and 9. The expression `8'd59` is ordinary binary decimal 59, stored as hexadecimal 3B; B is not a valid BCD digit. The final accepted code correctly compares the clock registers with `8'h59`.

Entry 123: complete rollover structure

The accepted design is a cascade of enable conditions: seconds always advance when ena is high; minutes advance only when old seconds are 59; hours advance only when old minutes and seconds are both 59.

```
always @(posedge clk) begin
  if (reset) begin
    second <= 8'h00; minutes <= 8'h00;
    hour   <= 8'h12; pmreg  <= 1'b0;
  end else if (ena) begin
    // One owner for seconds
    if (second == 8'h59)
      second <= 8'h00;
    else if (second[3:0] == 4'd9)
      second <= {second[7:4] + 4'd1, 4'd0};
    else
      second <= second + 8'd1;

    // Carry from seconds into minutes
    if (second == 8'h59) begin
      if (minutes == 8'h59)
        minutes <= 8'h00;
      else if (minutes[3:0] == 4'd9)
        minutes <= {minutes[7:4] + 4'd1, 4'd0};
      else
        minutes <= minutes + 8'd1;
    end

    // Carry from minutes into hours
    if (second == 8'h59 && minutes == 8'h59) begin
      if (hour == 8'h11) begin
        hour   <= 8'h12;
        pmreg <= ~pmreg;
      end else if (hour == 8'h12)
        hour <= 8'h01;
      else if (hour[3:0] == 4'd9)
        hour <= 8'h10;
      else
        hour <= hour + 8'd1;
    end
  end
end
```

Worked boundary transitions

Before enabled edge	After edge	Reason
09:59:59 AM	10:00:00 AM	BCD hour carry, no AM/PM toggle
11:59:59 AM	12:00:00 PM	11 becomes 12 and toggles PM
12:59:59 PM	01:00:00 PM	12 becomes 01 without toggling PM
11:59:59 PM	12:00:00 AM	11 becomes 12 and toggles back to AM

Points to remember

1. Reset establishes 12:00:00 AM. 2. If ena is low, every register holds. 3. Toggle AM/PM only on the 11-to-12 transition. 4. Use nonblocking assignments for every clocked state update; write `pmreg <= ~pmreg`, not a blocking assignment. 5. Each register should have one clear update section.

Source: [HDLBits - 12-hour BCD clock](#)